



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width
self.height
self.walls
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

```
self.opponents = []
```

The environment contains opponents that move independently and can affect the agent's performance through collisions.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The absolute, cumulative historical log of every single percept the agent has received from its initialization up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors (S)

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable

It does not know Exact locations of all food items, Locations of walls, Future

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

When `execute_action()` deducts points for hitting a wall, it updates the Performance Measure component of PEAS because the agent's score is being changed to reflect its performance in the environment.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

The metric should measure external outcomes because it shows whether the agent is achieving its goals. Penalizing wall collisions reflects the real impact of the agent's actions, while internal processing effort does not indicate success in the environment.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes partially observable because the toxic traps exist but are hidden from the agent's sensors, so the agent cannot observe the complete state of the environment.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The 'smells_toxin' sensor gives the agent information about nearby danger, allowing it to avoid toxic traps and make better decisions. This helps the agent maximize its score and behave more rationally.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World Exploit occurs when an agent exploits a poorly designed scoring system by repeatedly creating and cleaning messes to increase its score rather than achieving the intended goal efficiently.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING